

Docker Networks

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

Docker Network Architecture and Port Mapping

Docker Network Architecture: Core Concepts

- **Overview of Docker Networking:**

- ▶ Decouples containerised applications from the underlying physical infrastructure.
- ▶ Provides isolated, predictable network stacks for every container by default.

- **Key Architectural Pillars:**

- ▶ **Network Namespaces:** Complete isolation of network interfaces, routing tables, and port allocations.
- ▶ **Inter-Container Communication:** Secure, direct communication over software-defined virtual networks.
- ▶ **External Connectivity:** Controlled port binding and NAT mechanisms for public or LAN access.

Linux Network Namespaces and veth Pairs

- **Linux Network Namespaces:**

- ▶ Provides an isolated instance of the system network stack.
- ▶ Each container gets its own loopback interface, IP address, routing table, and firewall rules.

- **Virtual Ethernet Pairs (veth pairs):**

- ▶ Act as a virtual cable connecting the container to the host environment.
- ▶ **One end** sits inside the container's Network Namespace (e.g., eth0).
- ▶ **The other end** attaches to the host's virtual bridge (e.g., vethXYZ bound to docker0).

Traffic Path

Container (eth0) ↔ Virtual Pair (veth) ↔ Host Bridge (docker0)

The Container Network Model (CNM): Architecture

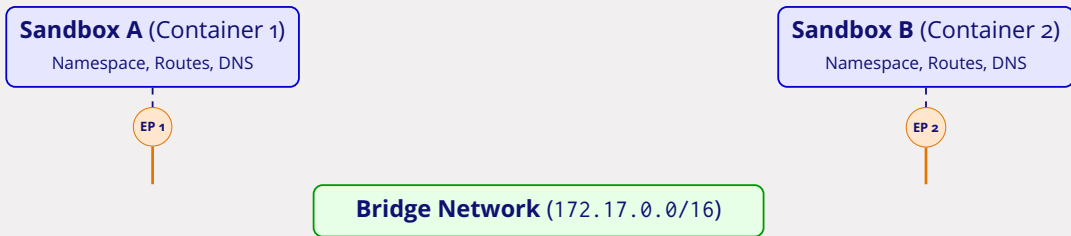
- **What is the CNM?**

- ▶ The design specification adopted by Docker to provide container networking across diverse infrastructure.
- ▶ Promotes complete abstraction: application containers remain agnostic of the underlying network driver.

- **The Three Core Constructs:**

- ▶ **Sandbox:** Contains the network configuration for a container (isolated network namespace, routing tables, and DNS settings).
- ▶ **Endpoint:** A virtual interface that joins a Sandbox to a Network.
- ▶ **Network:** A software-defined group of Endpoints that can communicate directly with one another.

CNM Topology: Sandbox, Endpoint, and Network



Key Architectural Rules

- A **Sandbox** can contain *multiple* Endpoints (connecting one container to multiple networks).
- An **Endpoint** belongs to exactly *one* Sandbox and *one* Network.

libnetwork and the IPAM Subsystem

- **The Role of libnetwork:**

- ▶ Docker's open-source Go library implementing the CNM specification.
- ▶ Manages driver registration, endpoint creation, and cross-host overlay networking.

- **IP Address Management (IPAM) Subsystem:**

- ▶ **Automated Allocation:** Assigns subnets, gateways, and individual container IP addresses dynamically.
- ▶ **Conflict Prevention:** Tracks allocated IPs across all running containers to prevent collisions.
- ▶ **Lifecycle Management:** Reclaims and returns IP addresses to the available pool when containers are removed.
- ▶ **Custom Subnets:** Allows administrators to define explicit network ranges and gateways using `-subnet` and `-gateway`.

Port Mapping and Traffic Binding: Concepts

- **The Isolation Challenge:**

- ▶ Container IPs are private (e.g., 172.17.0.x) and contained within the host's virtual bridge network.
- ▶ External clients or LAN users cannot route packets directly to a container IP.

- **The Port Mapping Solution (-p / -publish):**

- ▶ Binds a specific host network port to a port inside the container network namespace.
- ▶ Syntax: `-p <host_port>:<container_port>`

Port Mapping: Practical Syntax and Use Cases 1/2

- **Standard Public Binding:**

- ▶ Maps host port 8080 to container port 80 across all host network interfaces (0.0.0.0).

```
docker run -d -p 8080:80 nginx
```

- **Loopback / Localhost Only Binding:**

- ▶ Restricts access exclusively to the host machine for enhanced security (e.g., local databases).

```
docker run -d -p 127.0.0.1:5432:5432 postgres
```

Port Mapping: Practical Syntax and Use Cases 2/2

- **Dynamic Host Port Allocation:**

- ▶ Omits the host port, allowing Docker to assign an available ephemeral port automatically (typically in range 32768–60999).

```
docker run -d -p 80 nginx
```

- **UDP Traffic Forwarding:**

- ▶ Appends protocol flag to forward UDP traffic (defaults to TCP).

```
docker run -d -p 53:53/udp bind9
```

Under the Hood: How Port Forwarding Works

- **The Core Problem:**

- ▶ The host receives packets addressed to its own IP address and published port.
- ▶ These packets must be seamlessly translated and routed to the container's isolated network namespace.

- **Docker's Two-Pronged Approach:**

- ▶ **iptables NAT Rules (Kernel Space):** The primary, high-performance path for packet manipulation directly within the Linux kernel.
- ▶ **docker-proxy (User Space):** A lightweight helper process that handles fallback scenarios, local host loopback, and IPv6 bridging.

For maximum throughput, **kernel-space packet rewriting** does 99% of the work. `docker-proxy` acts as a safety net.

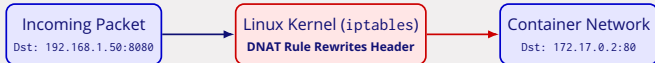
1. Kernel Space Routing: iptables and DNAT

- **Destination Network Address Translation (DNAT):**

- ▶ Docker creates custom rules in the DOCKER chain of the nat table.
- ▶ As packets arrive at host_ip:host_port, the kernel intercepts them and rewrites their destination header to container_ip:container_port.

- **Performance Advantage:**

- ▶ Executed inside **kernel space** — near-zero CPU overhead and maximum network throughput.



2. User Space Fallback: O docker-proxy

- **What is docker-proxy?**

- ▶ A dedicated user-space process spawned by the Docker daemon **for every published port** (e.g., one process per -p flag).
- ▶ Listens on the host port and manually proxies TCP/UDP streams to the container.

- **When is docker-proxy Used?**

- ▶ **Local Loopback Traffic:** When connecting from the host itself via 127.0.0.1 or localhost where kernel NAT rules might be bypassed.
- ▶ **IPv6 to IPv4 Translation:** Bridges incoming IPv6 host traffic to IPv4-only container networks.
- ▶ **Fallback Mode:** Ensures connectivity if iptables rules are restricted or disabled by system administrators.

Inspecting Docker Network Architecture in Action

Inspect Container Network Details:

```
# Check container IP and network endpoints  
docker inspect --format '{{json .NetworkSettings.Networks}}' my-app
```

Inspect Host Firewall Rules (iptables):

```
# View DNAT rules added by Docker daemon  
sudo iptables -t nat -L DOCKER -n -v
```

Verify Port Listening Status on Host:

```
# Identify docker-proxy processes listening on host ports  
ss -tulpn | grep 8080
```

Implementation, Best Practices

Practical Docker Networking and Administration

- **Transitioning from Architecture to Practice:**
 - ▶ We covered namespaces, iptables, and port mapping.
 - ▶ Now, we apply these concepts using the CLI and user-defined networks.
- **The Standard Operational Baseline:**
 - ▶ Approximately 90% of containerised workloads rely on the default **Bridge driver**.
 - ▶ User-defined bridges provide automatic container isolation and built-in DNS resolution.

User-Defined Networks and Automatic Service Discovery

- **Why Avoid the Default bridge Network?**

- ▶ The default bridge network lacks internal DNS; containers must communicate via static IP addresses.
- ▶ Unrelated containers on the default bridge can reach each other freely, posing security risks.

- **User-Defined Bridges (docker network create):**

- ▶ Provides complete network isolation between application stacks.
- ▶ Enables **Embedded Docker DNS**: containers discover each other simply by using their container name or alias.

Step 1 of 3: Provisioning an Isolated Network

```
docker network create my-app-net
```

- **Purpose:**

- ▶ Creates a new, isolated software-defined bridge network named my-app-net.
- ▶ Establishes a dedicated boundary for a specific application stack.

- **Why it is Crucial:**

- ▶ **Enables Embedded DNS:** Activates Docker's built-in DNS server (127.0.0.11) so containers on this network can resolve each other by name.
- ▶ **Network Isolation:** Prevents containers outside my-app-net from communicating with services inside it.

Step 2 of 3: Deploying the Internal Database

```
docker run -d --name postgres-db \  
  --network my-app-net \  
  -e POSTGRES_PASSWORD=secret postgres:15-alpine
```

- **Command Flag Breakdown:**

- ▶ `-d`: Runs the container in detached mode (in the background).
- ▶ `--name postgres-db`: Assigns a predictable hostname for internal DNS discovery.
- ▶ `--network my-app-net`: Attaches the container to our custom bridge network.
- ▶ `-e POSTGRES_PASSWORD=...`: Sets the required database admin password.

- **Security Design Note:**

- ▶ **No `-p` flag is used.** The database is intentionally **not** published to the host network, protecting it from external internet access.

Step 3 of 3: Deploying the Public-Facing Web API

```
docker run -d --name web-api \  
  --network my-app-net \  
  -p 8080:8080 \  
  -e DB_HOST=postgres-db \  
  my-web-api:1.0
```

- **Command Flag Breakdown:**

- ▶ `-network my-app-net`: Places the API on the same network as the database.
- ▶ `-p 8080:8080`: Publishes host port 8080 to expose the web service publicly.
- ▶ `-e DB_HOST=postgres-db`: Configures the application to reach the database using its **container name** instead of a hardcoded IP.

- **How it Works under the Hood:**

- ▶ When the API queries `postgres-db`, Docker's DNS resolves it directly to the database's private IP (e.g., `172.19.0.2`).

Network Lifecycle Management Commands

List Active Networks:

```
docker network ls
```

Connect or Disconnect Running Containers On-the-Fly:

```
# Attach an existing container to a second network
docker network connect monitoring-net web-api
```

```
# Detach a container from a network
docker network disconnect my-app-net web-api
```

Clean Up Unused Networks:

```
# Remove all custom networks not used by at least one container
docker network prune
```

Troubleshooting Network Issues

1. Inspect Network Topology and Attached IPs:

```
docker network inspect my-app-net
```

2. Verify Host Port Bindings with ss:

```
# Confirm if docker-proxy or kernel is listening on host port 8080  
ss -tulpn | grep 8080
```

3. Test Inter-Container Reachability:

```
# Run an ephemeral container to test DNS and connectivity  
docker run --rm -it --network my-app-net busybox ping -c 3 postgres-db
```

Docker Networking Best Practices Checklist

- **Always Use User-Defined Bridges:**
 - ▶ Never rely on the default bridge for production workloads.
 - ▶ Leverage embedded DNS for service-to-service communication instead of hardcoded IPs.
- **Follow the Principle of Least Privilege:**
 - ▶ Expose ports using `-p` **only** for public-facing edge containers (e.g., reverse proxies, web fronts).
 - ▶ Keep databases and internal APIs isolated strictly within private networks.
- **Explicit Binding Security:**
 - ▶ Bind sensitive services to `127.0.0.1:<port>` if they only need to be accessed from the local host machine.